

# Projektuppgift

*Programmering i TypeScript, DT208G*

**Projektuppgift**

Typescript Universitetet

**Yasmine Budak**



**Mittuniversitetet**  
MID SWEDEN UNIVERSITY

Campus Härnösand Universitetsbacken 1, SE-871 88. Campus Sundsvall Holmgatan 10, SE-851 70 Sundsvall.  
Campus Östersund Kunskapens väg 8, SE-831 25 Östersund.  
Phone: +46 (0)771 97 50 00, Fax: +46 (0)771 97 50 01.

**MITTUNIVERSITETET**

**Avdelningen för informationssystem och -teknologi**

**Författare:** Yasmine Budak, yabu2300@student.miun.se

**Utbildningsprogram:** Webbutveckling, 120 hp

**Huvudområde:** Datateknik

**Termin, år:** 02, 2026

## Sammanfattning

Denna rapport beskriver utvecklingen av en webbapplikation för ett fiktivt universitet, där användare kan söka bland kurser och skapa ett eget ram-schema. Applikationen är utvecklad med Angular som ramverk och TypeScript som programmeringsspråk. Syftet med projektet är att omsätta tidigare kunskaper från kursen i ett större sammanhängande projekt.

Webbplatsen hämtar kursdata från en extern JSON-fil som innehåller information om kurser från Mittuniversitetet. Användaren kan sortera, bläddra mellan, filtrera och välja kurser samt lägga till dessa i ett ram-schema. Ramschemat lagras i webbläsarens localStorage så att informationen sparas mellan sidladdningar.

Projektet innehåller flera komponenter, routing mellan undersidor samt services för hantering av data. Resultatet är en fungerande webbplats som uppfyller samtliga grundkrav för uppgiften.

# Innehållsförteckning

|                                  |           |
|----------------------------------|-----------|
| <b>Sammanfattning .....</b>      | <b>2</b>  |
| <b>Innehållsförteckning.....</b> | <b>3</b>  |
| <b>1 Introduktion .....</b>      | <b>4</b>  |
| <b>2 Teori .....</b>             | <b>6</b>  |
| <b>3 Metod .....</b>             | <b>8</b>  |
| <b>4 Konstruktion .....</b>      | <b>10</b> |
| <b>5 Resultat .....</b>          | <b>14</b> |
| <b>6 Slutsatser .....</b>        | <b>16</b> |
| <b>7 Källförteckning.....</b>    | <b>17</b> |

# 1 Introduktion

Syftet med detta projekt är att skapa en webbplats för ett fiktivt lärosäte där studenter kan söka efter och lista tillgängliga kurser samt skapa ett eget ramschema av en uppsättning av kurser. Problemet som ska lösas är hur man på ett strukturerat och användarvänligt sätt kan presentera stora mängder kursdata och samtidigt ge användaren möjlighet att välja och organisera kurser.

De problem som ska lösas är att användaren enkelt ska kunna:

- Se en lista med kurser
- Kunna sortera dessa kurser på kurskod, kursnamn, poäng, ämne.
- Filtrera kurserna på kurskod och kursnamn
- Lägga till och ta bort kurser i ett eget ramschema med tydligt gränssnitt som visar sammanlagda antal högskolepoäng
- Spara sitt schema

Projektet ska även visa att externa data kan läsas in, behandla detta och presenteras på skärm. Tillsammans ska TypeScript som programmeringsspråk och Angular som ramverk användas i en större applikation. Detta innebär att flera delar ska samverka, exempelvis komponenter, services och datahantering. Källkoden skall versionshanteras med Git, och den färdiga webbplatsen skall publiceras på en valfri webbhost.

Andra praktiska krav på utformningen av webbplatsen är:

- Webbssidan skall ha minst två undersidor, en för att söka och visa information om kurser - och en som visar skapat ramschema.
- Komponenter och routing skall användas.
- Minst två stycken services skall skapas, en för kursdata och en för hantering av ramschema.
- Det får inte vara möjligt att skapa dubletter av samma kurs i ramschemat.
- Listan med valda kurser skall uppdateras utan sidomladdning, och skapat ramschema skall lagras med hjälp av localStorage, och läsas in på nytt vid inladdning av webbsidan.
- Webbplatsen skall vara snygg, prydlig och ha ett välgenomarbetat helhetsintryck - och fungera väl på stora som små skärmar med bra responsiv design.
- Kurserna samt ramschemat skall presenteras på ett tydligt tillgängligt sätt.

- Att välja ut kurser från ämne - exempelvis; bara visa kurser som tillhör ämnet "Datateknik".
- Se antal kurser i aktuell sökning, exempelvis alla kurser, eller antal kurser i valt urval (ex hur många kurser det blir om enbart "Data-teknik"-kurser visas).

## 2 Teori

Angular är ett frontend-ramverk som används för att skapa moderna webb-applikationer. Ett ramverk innebär att det finns färdiga funktioner och strukturer som hjälper utvecklaren att bygga applikationer på ett mer organiserat sätt. Angular används för att skapa dynamiska webbplatser där innehållet kan förändras utan att hela sidan behöver laddas om. I detta projekt används Angular för att skapa undersidor, hantera data och bygga upp användargränssnittet. [1]

En viktig del i Angular är komponenter. Komponenter används för att dela upp webbplatsen i mindre delar. Varje komponent innehåller vanligtvis tre delar: HTML för struktur, CSS för design och TypeScript för logik och funktionalitet. Genom att använda komponenter blir projektet mer organiserat och lättare att underhålla. I detta projekt används exempelvis en komponent för kurslistan och en annan komponent för ramschemat. [3]

Routing är en funktion i Angular som används för att navigera mellan olika undersidor i en webbapplikation. Istället för att ladda om hela webbplatsen byts endast innehållet på sidan ut. Detta gör webbplatsen snabbare och ger en bättre användarupplevelse. I projektet används routing för att växla mellan sidan för kurser och sidan för ramschema. [4]

TypeScript är programmeringsspråket som används tillsammans med Angular. TypeScript bygger på JavaScript men innehåller fler funktioner, exempelvis datatyper och bättre strukturering av kod. Datatyper används för att bestämma vilken typ av information en variabel får innehålla, exempelvis text eller nummer. [5]

Services är en annan viktig del av Angular. Services används för att hantera data och funktioner som ska delas mellan flera komponenter. Genom att använda services kan logik separeras från användargränssnittet, vilket gör koden mer organiserad. I detta projekt används en service för att läsa in kursdata och en annan service för att hantera ramschemat. [6]

JSON står för JavaScript Object Notation och är ett vanligt format för att lagra och överföra data. JSON-filer innehåller information i textformat som är lätt för både människor och program att läsa. I projektet används en JSON-fil som innehåller information om universitetets kurser, exempelvis kurskod, kursnamn och antal högskolepoäng. [7]

HttpClient är en funktion i Angular som används för att hämta data från externa filer eller servrar. I detta projekt används HttpClient för att läsa in kursinformationen från JSON-filen och göra datan tillgänglig i applikationen. [8]

LocalStorage är en funktion som finns i webbläsaren och används för att lagra data lokalt på användarens dator. Informationen sparas även om webb-

sidan laddas om eller om webbläsaren stängs. I projektet används LocalStorage för att spara användarens ramschema så att kurserna finns kvar vid nästa besök. [9]



### 3 Metod

I detta projekt kommer flera olika tekniker, program och arbetsmetoder att användas för att utveckla webbapplikationen. Dessa metoder kommer att användas för att lösa de problem som identifierats i introduktionen, exempelvis hur kursdata ska hanteras, hur användaren ska kunna skapa ett eget ramschema samt hur data ska sparas mellan sidomladdningar.

Angular kommer att användas som ramverk för utvecklingen av webbplatsen. Genom att använda Angular kommer applikationen att kunna delas upp i mindre delar, så kallade komponenter. Detta kommer att göra projektet mer organiserat och lättare att underhålla. Angular kommer även att användas för routing mellan undersidor och för att hantera dataflöden i applikationen. [1]

TypeScript kommer att användas som programmeringsspråk i projektet. Detta kommer att göra koden mer strukturerad och enklare att läsa. TypeScript kommer även att bidra till att upptäcka fel tidigare under utvecklingen, vilket kan minska risken för buggar i applikationen. [2]

HTML kommer att användas för att bygga upp webbplatsens struktur och innehåll. Med hjälp av HTML kommer olika element såsom rubriker, knappar, tabeller och formulär att skapas. CSS kommer sedan att användas för att designa och styla webbplatsen. CSS kommer att användas för att skapa layout, färger, mellanrum och responsiv design så att webbplatsen fungerar på både stora och små skärmar.

För att skapa en responsiv design kommer flexbox och media queries att användas i CSS. Detta kommer att göra att webbplatsen automatiskt anpassar sig beroende på användarens skärmstorlek. Målet är att webbplatsen ska fungera lika bra på mobiltelefoner som på datorer.

Visual Studio Code kommer att användas som utvecklingsmiljö under projektets gång. Programmet kommer att användas för att skriva och redigera kod i TypeScript, HTML och CSS. Visual Studio Code innehåller flera funktioner som underlättar utvecklingsarbetet, exempelvis syntaxmarkering samt felsökning.

För versionshantering kommer Git att användas. Git kommer att användas för att spara olika versioner av projektet under utvecklingens gång. Detta kommer att göra det möjligt att följa förändringar i koden och återställa tidigare versioner vid behov. GitHub kommer även att användas för att publicera projektets källkod online.

För att hantera externa data kommer Angulars HttpClient att användas. HttpClient kommer att användas för att läsa in JSON-filen som innehåller kursinformation. Informationen kommer därefter att bearbetas och presenteras på webbplatsen. Genom denna metod kommer kursdata kunna laddas dynamiskt i stället för att skrivas direkt i koden.

JSON-format kommer att användas eftersom det är ett vanligt format för lagring och överföring av data. JSON-filen innehåller information om kurser, exempelvis kurskod, kursnamn, ämne och antal högskolepoäng.

För lagring av användarens ramschema kommer LocalStorage att användas. LocalStorage är en funktion i webbläsaren som gör det möjligt att lagra data lokalt på användarens dator. Denna metod kommer att användas för att användarens valda kurser ska sparas även om webbsidan laddas om eller stängs ner. Informationen kommer att sparas i JSON-format och läsas in automatiskt när webbplatsen öppnas igen.

Canva kommer att användas för att skapa webbplatsens logotyp. Programmet kommer att användas eftersom det är enkelt att skapa designmaterial med och gör det möjligt att ta fram en tydlig visuell identitet för webbplatsen. Detta kommer att bidra till en mer genomarbetad grafisk profil.

Under projektets gång kommer kontinuerlig testning att genomföras för att säkerställa att funktionerna fungerar korrekt. Exempelvis kommer filtrering, sortering och lagring av data att testas löpande under utvecklingen. Webbplatsen kommer även att testas på olika skärmstorlekar för att säkerställa att designen är responsiv.

Slutligen kommer webbplatsen att publiceras på en publik webbhost kallad Netlify för att göra applikationen tillgänglig online. Publiceringen kommer att göra det möjligt att testa webbplatsen i en verklig miljö och säkerställa att alla funktioner fungerar korrekt även efter deployment.

## 4 Konstruktion

Konstruktionen av webbapplikationen började med att ett nytt Angular-projekt skapades med hjälp av Angular CLI i Visual Studio Code. Efter att projektet skapats installerades de paket och moduler som behövdes för utvecklingen, exempelvis Angular Router för navigering mellan sidor och HttpClient för inläsning av externa data.

När projektets grundstruktur var färdig började arbetet med att planera applikationens uppbyggnad. Webbplatsen delades upp i flera komponenter för att skapa en tydlig struktur och göra koden mer organiserad. Två huvudsakliga undersidor skapades i projektet, en för kurslistan och en för användarens ramschema. För att hantera navigeringen mellan dessa sidor användes Angular Routing. I filen `app.routes.ts` definierades routing-tabellen där användaren automatiskt skickas vidare till kurssidan vid start av applikationen. Routing användes för att skapa en mer modern användarupplevelse där innehåll kan bytas utan att hela sidan behöver laddas om.

Efter att routing implementerats skapades huvudkomponenten App som fungerar som applikationens grundlayout. I `app.html` skapades en navigeringsmeny med länkar till undersidorna Kurser och Ramschema. Där placerades även webbplatsens logotyp samt en huvudrubrik för webbplatsen. För att visa rätt komponent beroende på vald route användes Angulars `router-outlet`. På detta sätt kunde olika delar av applikationen visas dynamiskt utan sidomladdning.

För att skapa en tydlig grafisk profil skapades en egen logotyp och en favicon i Canva. Dessa exporterades som en png och placerades i projektets `image-mapp`. Därefter användes dessa både som logotyp i applikationen och som favicon i `index.html`. Syftet med detta var att skapa ett mer professionellt och genomarbetat helhetsintryck.

Nästa steg i konstruktionen var att skapa kurshanteringen. För detta skapades först ett interface i filen `courseModel.ts`. Interfacet definierade strukturen för varje kursobjekt och innehöll egenskaper såsom kurskod, kursnamn, ämne, poäng och kursplan. Genom att använda ett interface i TypeScript kunde datan hanteras på ett tydligare och säkrare sätt eftersom alla kursobjekt måste följa samma struktur.

Efter detta skapades `CourseService` som ansvarar för att läsa in kursdata från JSON-filen `miun_courses.json`. JSON-filen placerades i projektets `public-mapp` för att kunna hämtas av applikationen. För att läsa in filen användes Angulars `HttpClient`. Metoden `getCourses` returnerar en `Observable` med

kursdata, vilket gjorde det möjligt att arbeta med asynkron datahämtning. Denna lösning gjorde att kursinformationen kunde laddas dynamiskt istället för att skrivas direkt i applikationen.

När datahanteringen för kurser var färdig skapades `CoursesComponent` som ansvarar för att visa kurslistan för användaren. I komponentens constructor anropades `CourseService` för att hämta alla kurser från JSON-filen. Kursdata sparades sedan i arrayerna `courses` och `filteredCourses`. Samtidigt skapades en lista med unika ämnen genom att använda JavaScripts `Set`-funktion. Dessa ämnen användes senare i filtreringsfunktionen för att användaren skulle kunna välja att endast visa kurser inom ett specifikt område.

I `courses.html` byggdes användargränssnittet upp med hjälp av HTML och Angular-direktiv. Ett sökfält skapades där användaren kan skriva in text för att filtrera kurser. Databindning implementerades med hjälp av `ngModel`, vilket gjorde att användarens inmatning automatiskt kopplades till variabeln `searchTerm` i TypeScript-koden. Ett `select-element` användes även för ämnesfiltrering där användaren kan välja ett specifikt ämne från en lista.

För att hantera filtreringen skapades metoden `searchCourses` i `courses.ts`. Metoden filtrerar kurslistan genom att kontrollera om användarens sökord matchar kursnamn, kurskod, ämne eller kursplan. Samtidigt kontrolleras även om användaren valt ett specifikt ämne i dropdown-menyn. Om ett ämne valts visas endast kurser inom det området. När filtreringen genomförs återställs även sidnumret till första sidan för att undvika problem vid paginering.

Sorteringsfunktionen implementerades genom metoden `sortBy`. När användaren klickar på en tabellrubrik skickas namnet på kolumnen till funktionen. Funktionen kontrollerar om samma kolumn redan används för sortering och växlar då mellan stigande och fallande ordning. Därefter sorteras kurslistan dynamiskt med hjälp av JavaScripts `sort`-funktion. För textvärden används `toLowerCase` för att göra sorteringen okänslig för skiftläge.

Som extra funktionalitet implementerades även paginering. Syftet med detta var att förbättra användarupplevelsen när många kurser visas samtidigt. I komponenten skapades variabler för aktuell sida och antal objekt per sida. Genom `get`-funktionen `paginatedCourses` beräknas vilka kurser som ska visas på den aktuella sidan. Knappar för nästa och föregående sida skapades i HTML-filen och kopplades till funktionerna `nextPage` och `previousPage`. På detta sätt kunde användaren navigera mellan olika sidor i kurslistan.

För att användaren skulle kunna skapa ett eget ramschema utvecklades `ScheduleService`. Denna service ansvarar för att lagra, hämta och uppdatera

användarens valda kurser. LocalStorage användes för lagring eftersom uppgiften krävde att ramschemat skulle sparas även efter omladdning av webbsidan. När användaren lägger till kurs hämtas först den aktuella listan från localStorage. Därefter kontrolleras om kursen redan finns i schemat genom att jämföra kurskoder. Om kursen inte redan existerar sparas den nya listan tillbaka till localStorage i JSON-format. Denna kontroll implementerades för att förhindra dubletter i ramschemat.

Efter detta skapades ScheduleComponent som ansvarar för att visa användarens ramschema. När komponenten laddas hämtas alla sparade kurser från ScheduleService. Kurserna visas därefter i en tabell där användaren även kan ta bort kurser från sitt schema. När en kurs tas bort uppdateras både komponentens data och localStorage direkt utan att sidan behöver laddas om.

I ScheduleComponent implementerades även funktionen totalPoints som beräknar den sammanlagda mängden högskolepoäng för de valda kurserna. Funktionen använder reduce-metoden för att summera poängen för varje kurs i schemat. Resultatet visas sedan direkt på sidan.

För design och layout användes CSS i samtliga komponenter. Fokus låg på att skapa en enkel, tydlig och användarvänlig design. Tabellerna stylades med färger, mellanrum och hover-effekter för att förbättra läsbarheten. Knappar gavs tydliga färger och avrundade hörn för ett modernare utseende.

En viktig del av konstruktionen var även att göra webbplatsen responsiv. Detta genomfördes med hjälp av media queries i CSS. När skärmstorleken blir mindre än 768 pixllar förändras tabellernas layout automatiskt. Tabellhuvudet döljs och varje rad visas i stället som ett separat block där data-label används för att visa rubriker framför varje värde. Denna lösning gjorde att tabellerna blev betydligt mer lättlästa på mobiltelefoner och mindre skärmar.

Under hela utvecklingsprocessen användes Git för versionshantering. Genom Git kunde olika versioner av projektet sparas kontinuerligt under arbetsgång. Detta gjorde det möjligt att återställa tidigare versioner vid problem och gav bättre kontroll över projektets utveckling. Projektet laddades även upp till GitHub för att uppfylla uppgiftens krav på publik versionshantering.

När utvecklingen var färdig publicerades webbplatsen på Netlify. Publiceringen gjorde webbplatsen publikt tillgänglig online och gjorde det möjligt att testa applikationen i en verklig miljö. Innan publiceringen genomfördes flera tester i olika webbläsare på olika skärmar för att säkerställa att alla

funktioner fungerade korrekt, exempelvis routing, filtrering, paginering och localStorage. Genom dessa tester kunde eventuella fel identifieras och rättas innan den slutliga versionen publicerades.

## 5 Resultat

Resultatet av projektet blev en fungerande webbapplikation för ett fiktivt universitet där användaren kan söka bland kurser och skapa ett eget ram-schema. Webbplatsen utvecklades med Angular som ramverk och TypeScript som programmeringsspråk enligt uppgiftens krav. Samtliga grundkrav för projektet uppfyllades och webbplatsen fungerar som planerat.

Applikationen består av två huvudsakliga undersidor. Den första sidan används för att visa och hantera kurslistan medan den andra sidan visar användarens ramschema. Navigationen mellan sidorna fungerar genom Angular Routing och sker utan att hela sidan behöver laddas om. Detta bidrar till en snabbare och mer modern användarupplevelse.

Kurslistan visar information om varje kurs i tabellformat. Användaren kan sortera kurser efter kurskod, kursnamn, ämne och antal högskolepoäng. Sorteringen fungerar både i stigande och fallande ordning. Filtreringsfunktionen gör det möjligt att söka efter kurser genom att skriva in text i ett sökfält. Filtreringen söker bland annat efter kursnamn, kurskod och ämne. Det går även att filtrera kurser utifrån specifika ämnen genom en dropdown-meny.

Webbplatsen visar dessutom hur många kurser som visas i den aktuella sökningen. Detta gör det enklare för användaren att få en överblick över resultatet efter filtrering och sökning.

En viktig del av projektet var möjligheten att skapa ett eget ramschema. Användaren kan lägga till kurser direkt från kurslistan genom att trycka på knappen "Lägg till". De valda kurserna visas därefter på sidan för ramschemat. Systemet kontrollerar även att samma kurs inte kan läggas till flera gånger, vilket förhindrar dubletter.

Ramschemat lagras med hjälp av LocalStorage. Detta innebär att användarens valda kurser sparas även om sidan laddas om eller webbläsaren stängs. När webbplatsen öppnas igen läses datan automatiskt in från LocalStorage och visas på nytt. Användaren kan även ta bort kurser från sitt ramschema, vilket direkt uppdaterar både gränssnittet och den sparade datan.

Applikationen visar även den totala mängden högskolepoäng för de kurser som lagts till i ramschemat. Detta gör det enklare för användaren att planera sitt schema och få en tydlig överblick över studieomfattningen.

Som extra funktionalitet implementerades paginering i kurslistan. Funktionen delar upp kurserna på flera sidor och gör det möjligt att navigera mellan dem med hjälp av knappar för nästa och föregående sida. Detta förbättrar användarupplevelsen och sidobelastningen genom att minska mängden information som visas samtidigt.

Webbplatsen anpassades även för olika skärmstorlekar genom responsiv design. På mindre skärmar förändras tabellernas layout så att innehållet blir mer lättläst på mobiltelefoner och surfplattor. Detta gör att webbplatsen fungerar både på stora och små enheter.

Den grafiska profilen och gränssnittet är sammanhängande och har fokus på tydlighet samt pedagogik.

Projektets källkod versionshanterades med Git och publicerades på GitHub. Den färdiga webbplatsen publicerades även på Netlify och blev därmed tillgänglig online. Testning genomfördes kontinuerligt under utvecklingen och samtliga krav är uppnådda och problem lösta.



## 6 Slutsatser

Projektet har varit lärorikt och gett en djupare förståelse för hur större webbapplikationer byggs upp med Angular och TypeScript. Genom projektet har tidigare kunskaper från kursens olika moment kunnat användas tillsammans i en större och mer sammanhängande lösning.

En av de viktigaste erfarenheterna från projektet var arbetet med komponentbaserad utveckling i Angular. Genom att dela upp webbplatsen i mindre komponenter blev det enklare att organisera projektet och hålla koden strukturerad. Arbetet med services gav även en bättre förståelse för hur data kan delas och hanteras mellan olika delar av en applikation.

En annan viktig erfarenhet var användningen av LocalStorage för att lagra användardata. Funktionen gjorde det möjligt att skapa en mer användbar webbplats där användarens val sparas även efter omladdning av sidan. Även arbetet med HttpClient och inläsning av JSON-data gav en bättre förståelse för hur externa data kan hämtas och presenteras dynamiskt i Angular.

Under projektets gång uppstod vissa utmaningar, framför allt kring filterning, sortering och paginering av kursdata. Det krävdes planering samt flera försök för att funktionerna skulle fungera tillsammans utan konflikter. Genom testning och felsökning kunde problemen lösas.

Projektet hade även kunnat vidareutvecklas med fler funktioner. Exempelvis hade det varit möjligt att skapa inloggning för användare eller att lägga till fler undersidor med information om universitetet. Det hade även varit möjligt att utveckla designen ytterligare med fler grafiska element och animationer.

Sammanfattningsvis uppfyllde projektet de mål och krav som ställdes i uppgiften. Webbapplikationen fungerar som planerat och innehåller både grundkrav och extra funktionalitet i form av paginering. Projektet har gett mer praktisk erfarenhet av Angular och TypeScript.

## Källförteckning

- [1] Team A. What is Angular? • Angular [Internet]. Angular.dev. Angular; 2026 [hämtad 2026 Maj 05]. Tillgänglig: <https://angular.dev/overview>
- [2] The starting point for learning TypeScript [Internet]. typescriptlang. 2026. [hämtad 2026 Maj 05]. Tillgänglig: <https://www.typescriptlang.org/docs/>
- [3] Angular [Internet]. angular.dev. [hämtad 2026 Maj 05]. Tillgänglig: <https://angular.dev/guide/components>
- [4] Angular [Internet]. Angular.dev. Angular; 2024. [hämtad 2026 Maj 05]. Tillgänglig: <https://angular.dev/guide/routing>
- [5] Handbook - The TypeScript Handbook [Internet]. www.typescriptlang.org. [hämtad 2026 Maj 05]. Tillgänglig: <https://www.typescriptlang.org/docs/handbook/intro.html>
- [6] Angular [Internet]. angular.dev. [hämtad 2026 Maj 05]. Tillgänglig: <https://angular.dev/guide/di>
- [7] Working with JSON - Learn web development | MDN [Internet]. MDN Web Docs. 2024. [hämtad 2026 Maj 05]. Tillgänglig: [https://developer.mozilla.org/en-US/docs/Learn\\_web\\_development/Core/Scripting/JSON](https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/Scripting/JSON)
- [8] Team A. HTTP Client • Overview • Angular [Internet]. Angular.dev. Angular; 2025. [hämtad 2026 Maj 05]. Tillgänglig: <https://angular.dev/guide/http>
- [9] Mozilla. Window.localStorage [Internet]. MDN Web Docs. 2019. [hämtad 2026 Maj 05]. Tillgänglig: <https://developer.mozilla.org/en-US/docs/Web/API/Window/localStorage>